

요구사항 정의서

서비스 목표, 사용자 요구사항, 시스템/비기능 요구사항, 데모 성공 기준

프로젝트

YumYum

기준 커밋

a963b9d

생성일

2026. 06. 25.

1. 프로젝트 개요

YumYum은 사용자의 냉장고 재고를 기반으로 유통기한을 관리하고, 식단 기록과 레시피 추천을 연결하는 식생활 관리 서비스이다. 사용자는 OAuth 로그인 후 온보딩을 완료하고, 냉장고 재고를 수기/OCR/바코드 방식으로 등록한다. 서비스는 마감 임박 재료를 알려주고, 보유 재료를 활용한 레시피를 추천하며, 식단 기록을 통해 일일 영양 요약を提供한다.

2. 핵심 목표

- 냉장고 재고를 빠르게 등록하고 유통기한을 놓치지 않게 한다.
- 보유 재료 중심의 레시피 추천으로 음식물 낭비를 줄인다.
- 식단 기록과 영양 요약을 통해 사용자가 섭취 현황을 확인하게 한다.
- 관리자 화면에서 공지, 배치, API 사용량을 확인해 운영 안정성을 높인다.
- 외부 API는 `mock/live/fallback` 모드로 분리해 데모 환경에서도 중단 없이 동작하게 한다.

3. 사용자와 문제 정의

사용자	문제	YumYum 해결 방식
1인 가구/자취생	냉장고에 무엇이 남았는지 기억하기 어렵다.	재고 목록과 유통기한 상태를 대시보드에서 요약한다.
바쁜 학습자/직장인	식재료를 사두고도 활용하지 못해 버리는 일이 생긴다.	마감 임박 재료를 알림과 추천 레시피에 우선 반영한다.
식단을 기록하는 사용자	먹은 음식과 보유 식재료 데이터가 분리되어 있다.	식단 기록과 냉장고 재고 기반 추천을 한 서비스 흐름으로 연결한다.
운영자/PM	데모 중 외부 API 장애나 키 누락으로 시연이 멈출 수 있다.	<code>mock/live/fallback</code> 모드와 관리자 사용량 화면으로 안정성을 확보한다.

4. 사용자 요구사항

ID	요구사항	우선순위	수용 기준
UR-01	사용자는 Google/Naver/Kakao OAuth로 로그인할 수 있다.	High	로그인 후 신규 사용자는 온보딩 화면으로 이동한다.
UR-02	사용자는 성별, 생년월일, 키, 몸무게, 활동량 등 기본 정보를 입력할 수 있다.	High	온보딩 완료 후 대시보드에 접근할 수 있다.
UR-03	사용자는 냉장고 재고를 수기로 여러 개 등록할 수 있다.	High	등록 후 목록과 상세 화면에서 확인할 수 있다.
UR-04	사용자는 영수증/OCR 또는 바코드 이미지로 재고 후보를 분석할 수 있다.	Medium	분석 결과가 다중 입력 폼에 매핑된다.
UR-05	사용자는 재고 목록을 조회, 수정, 삭제할 수 있다.	High	변경 결과가 목록과 대시보드 통계에 반영된다.
UR-06	사용자는 음식명을 검색해 식단 기록에 추가할 수 있다.	High	저장 후 일일 요약 카드와 기록 목록이 갱신된다.
UR-07	사용자는 보유 재료 기반 레시피 추천을 받을 수 있다.	High	mock 모드에서도 추천 결과가 표시되고 최근 추천 이력이 저장된다.
UR-08	사용자는 유통기한/공지 알림을 조회하고 읽음 처리할 수 있다.	Medium	읽음 처리 후 안 읽은 알림 수가 갱신된다.
UR-09	관리자는 로그인 후 대시보드, API 사용량, 배치, 공지, 감사 로그를 확인할 수 있다.	Medium	<code>/admin</code> 에서 각 탭이 표시되고 주요 조작이 가능하다.

5. 시스템 요구사항

ID	요구사항	수용 기준
SR-01	FrontEnd, BackEnd, MySQL, OCR 서비스를 Docker Compose로 실행한다.	<code>docker compose up --build</code> 기준 각 서비스가 기동된다.
SR-02	민감값은 Git에 커밋하지 않고 환경변수로 주입한다.	<code>application.yml</code> 은 비밀값 없는 기본값만 포함한다.
SR-03	외부 AI/OCR/식품안전나라 API는 통합 모드로 제어한다.	<code>INTEGRATION_MODE=mock/live/fallback</code> 을 지원한다.
SR-04	백엔드는 공통 응답 포맷과 예외 처리를 사용한다.	성공/실패 응답이 일관된 형태로 내려간다.
SR-05	인증 요청은 JWT 기반으로 보호한다.	인증 실패 시 401/403이 반환되고 FE는 세션 만료를 처리한다.

6. 비기능 요구사항

- 데모 안정성: 외부 API 장애 또는 키 미설정 상황에서도 mock/fallback 결과로 핵심 화면을 시연할 수 있어야 한다.
- 사용성: 재고 등록, 식단 기록, 추천 생성 흐름은 한 화면에서 이어져야 한다.
- 유지보수성: 도메인별 controller/service/dao/entity/dto 구조를 유지한다.
- 확장성: 외부 API 호출은 provider/client 계층으로 분리해 실제 API와 mock 구현을 교체할 수 있어야 한다.
- 보안: OAuth client secret, API key, JWT secret은 문서/코드/캡처에 노출하지 않는다.

7. 핵심 품질 기준

기준	설명	확인 방법
부팅 가능성	fresh clone과 Docker 환경에서 서비스가 기동되어야 한다.	<code>docker compose config --quiet</code> , Docker 실행 확인
빌드 가능성	FE/BE가 각각 빌드 또는 컴파일되어야 한다.	<code>npm run build</code> , <code>mvn -DskipTests compile</code>
발표 안정성	외부 API가 없어도 추천/OCR 흐름 설명과 화면 캡처가 가능해야 한다.	<code>INTEGRATION_MODE=mock</code> 기준 화면 캡처
개인정보 보호	캡처와 문서에 실제 이메일, 토큰, secret이 없어야 한다.	문서 검색과 시각 검수
사용 흐름 완결성	로그인부터 관리자 화면까지 발표 시나리오가 끊기지 않아야 한다.	수동 시연 체크리스트

8. 데모 성공 기준

- 1. 실제 OAuth 로그인 후 신규 사용자 온보딩을 완료한다.
- 2. 냉장고에 식재료를 수기 등록하고 목록/상세/수정/삭제를 확인한다.
- 3. 식단 기록에서 김밥 을 검색해 저장하고 일일 요약 변화를 확인한다.
- 4. 레시피 추천을 생성하고 결과를 레시피 화면에서 확인한다.
- 5. 알림 목록을 조회하고 읽음 처리한다.
- 6. 관리자 화면에서 대시보드, 공지, 배치, API 사용량 화면을 확인한다.

9. 범위와 제외 사항

구분	이번 제출 범위	이후 개선 범위
사용자	단일 사용자 데모 흐름	다중 사용자 동시 접속, 운영 모니터링
추천	mock/live/fallback provider 구조와 저장 이력	개인화 모델 고도화, 장기 추천 품질 평가
인증	OAuth 로그인과 JWT 기반 API 보호	HttpOnly cookie 전환, 장기 토큰 운영 정책
배포	Docker Compose 기반 로컬/데모 실행	공개 URL, HTTPS, OAuth redirect 운영값
테스트	빌드/컴파일/수동 시연 중심	E2E 자동화, 80% 이상 커버리지

Use-Case 다이어그램

사용자, 관리자, 외부 API actor와 핵심 use-case 흐름

프로젝트

YumYum

기준 커밋

a963b9d

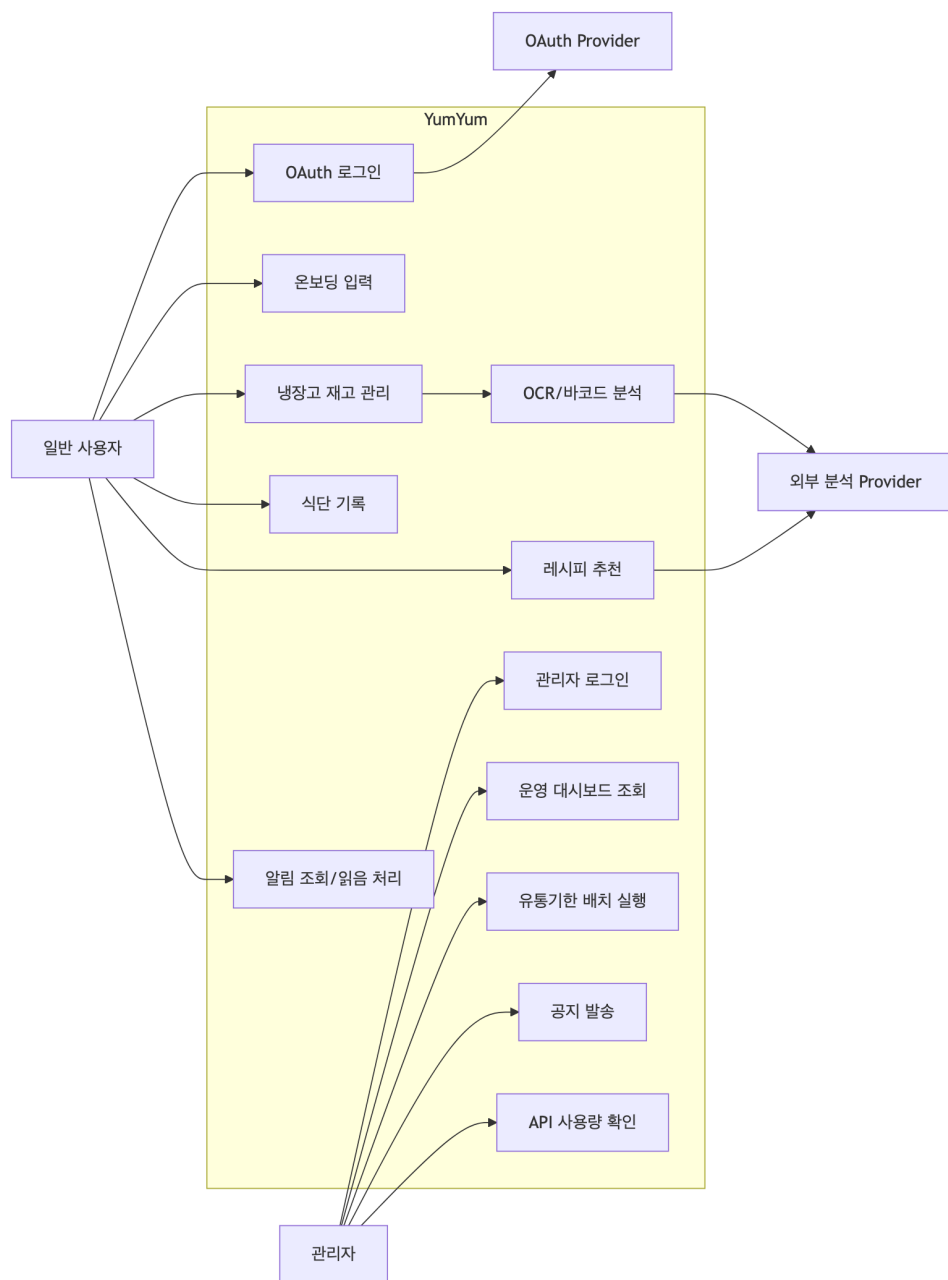
생성일

2026. 06. 25.

주요 액터

- 일반 사용자: 냉장고, 식단, 추천, 알림 기능을 사용하는 사용자
- 관리자: 서비스 운영 상태를 확인하고 공지/배치 작업을 수행하는 운영자
- OAuth Provider: Google, Naver, Kakao 로그인 제공자
- 외부 분석 Provider: Pororo OCR, GMS/GPT, 식품안전나라 API

Use-Case Mermaid 원본



Use-Case 다이어그램 다이어그램 1

주요 시나리오

시나리오	흐름
신규 사용자 시작	OAuth 로그인 → 온보딩 입력 → 대시보드 진입
냉장고 등록	재고 추가 → 수기/OCR/바코드 선택 → 입력값 확인 → batch 저장
식단 기록	음식 검색 → 섭취량 입력 → 저장 → 일일 요약 갱신
레시피 추천	냉장고 재고 조회 → 추천 생성 → 추천 이력 저장 → 레시피 카드 표시
유통기한 알림	배치 실행 → 임박/만료 재고 조회 → 알림 생성 → 사용자 조회
관리자 운영	관리자 로그인 → 지표 확인 → 배치 수동 실행 → 공지 발송 → 감사 로그 확인

Use-Case 상세 명세

Use-Case	선행 조건	기본 흐름	예외/대체 흐름	완료 조건
OAuth 로그인	사용자가 로그인 화면에 접근	provider 선택 → OAuth 인증 → JWT 수신 → 앱 진입	신규/Guest면 온보딩으로 이동	사용자 화면 접근 가능
냉장고 재고 등록	로그인 완료	등록 방식 선택 → 상품 정보 입력/분석 → batch 저장	OCR/바코드 실패 시 수기 입력 유지	목록에 신규 재고 표시
식단 기록 저장	로그인 완료, 음식 DB 존재	음식 검색 → 수량 입력 → 저장	검색 결과 없음이면 다른 검색어 안내	기록 목록과 영양 요약 갱신
레시피 추천 생성	로그인 완료, 재고 1개 이상	재고 조회 → provider 호출 → 추천 저장 → 화면 표시	live 실패 시 fallback/mock 결과 표시	추천 카드와 최근 추천 이력 생성
알림 읽음 처리	알림 존재	알림 목록 조회 → 개별/전체 읽음 처리	네트워크 실패 시 오류 메시지 표시	unread badge 감소
관리자 공지 발송	관리자 로그인	공지 입력 → 전체 사용자 대상 알림 생성 → 이력 저장	권한 만료 시 관리자 로그인으로 이동	공지 이력과 감사 로그 생성

발표용 설명 포인트

- 일반 사용자 use-case는 “냉장고 데이터가 식단/추천/알림으로 확장되는 흐름”을 강조한다.
- 관리자 use-case는 “행사 데모와 운영 안정성을 PM이 직접 확인할 수 있는 장치”로 설명한다.
- 외부 Provider는 직접 화면에 드러나는 기능보다, 장애를 흡수하는 구조적 안전장치로 소개한다.

클래스 다이어그램

도메인 계층, provider/client 경계, mock/live/fallback 협력 구조

프로젝트

YumYum

기준 커밋

a963b9d

생성일

2026. 06. 25.

설계 관점

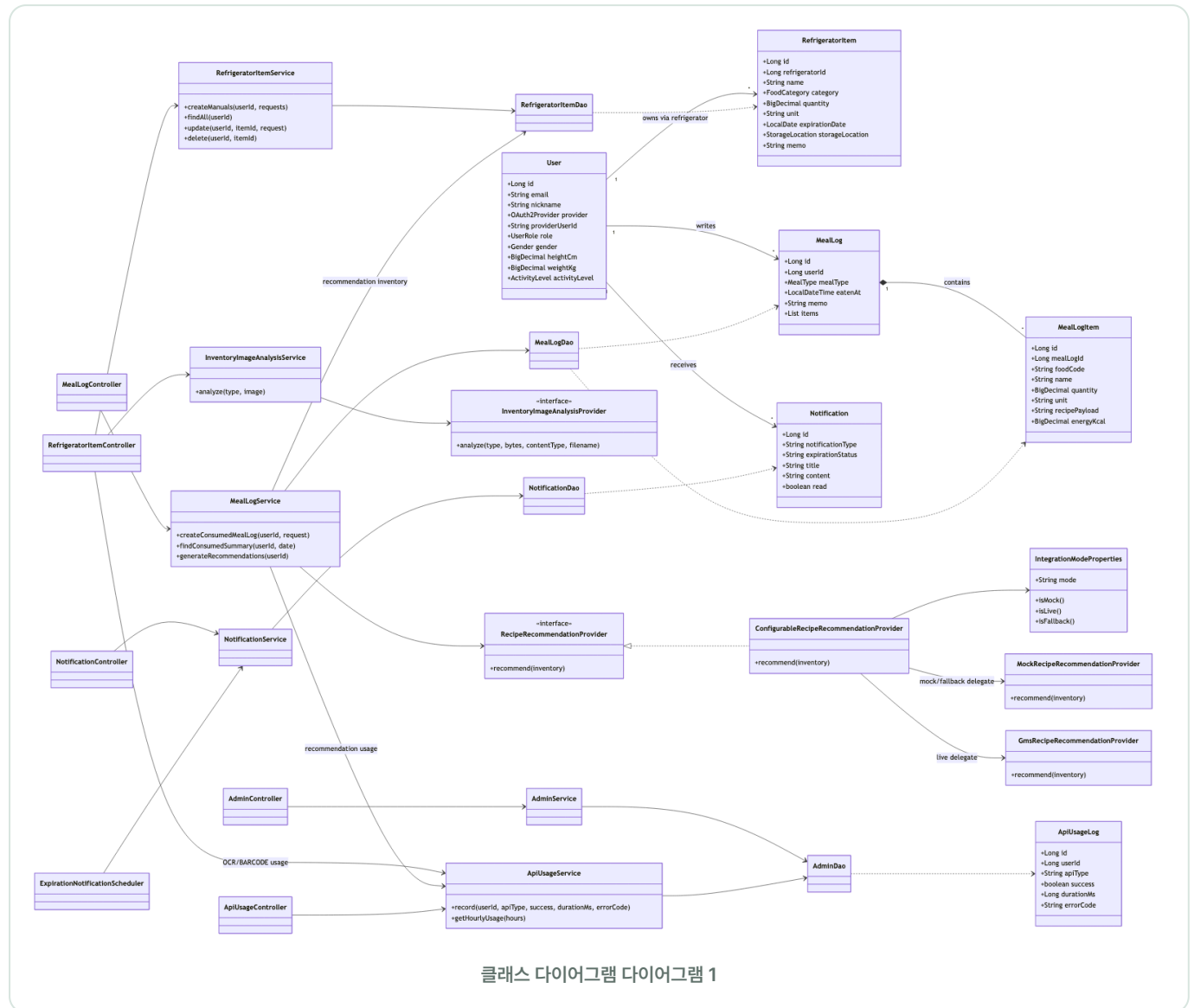
YumYum 백엔드는 도메인별 `controller -> service -> dao/entity` 구조를 따른다. 외부 연동은 provider/client 계층으로 분리해 `mock`, `live`, `fallback` 모드를 선택할 수 있게 했다.

이 문서는 전체 클래스의 기계적 나열이 아니라, 시연 MVP의 핵심 사용자 흐름을 설명하는 제출용 클래스 다이어그램이다. DTO/Enum, Lombok getter/setter, MyBatis XML mapper, 단순 request/response record는 가독성을 위해 일부 생략했다.

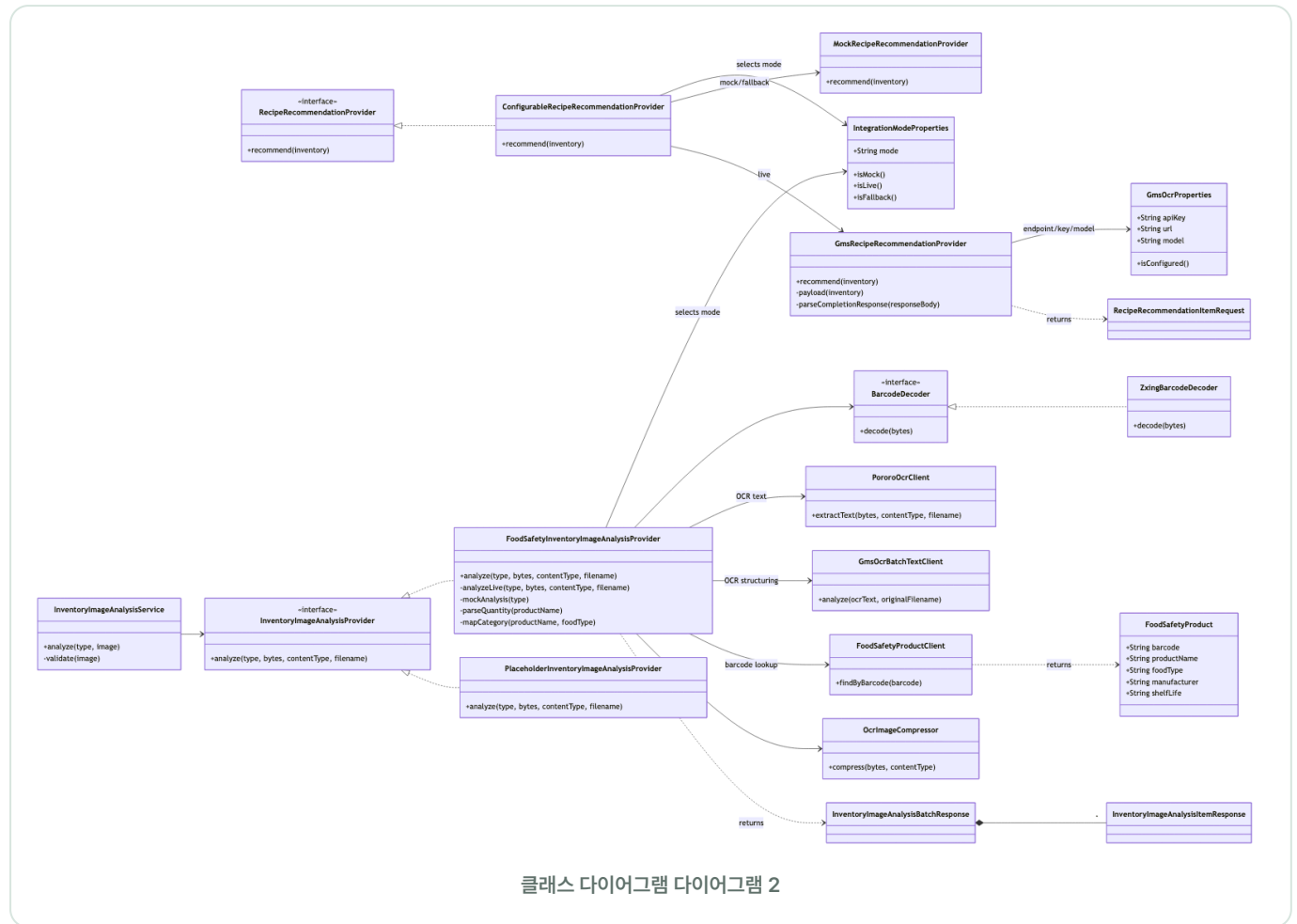
다이어그램 구성

파일	용도
<code>docs/final-report/04-assets/diagrams/class-diagram.mmd</code>	제출 문서 본문에 넣을 핵심 클래스 다이어그램
<code>docs/final-report/04-assets/diagrams/class-diagram-integration-detail.mmd</code>	발표에서 mock/live/fallback과 OCR/바코드 연동을 설명할 보조 다이어그램

핵심 클래스 다이어그램



외부 연동 상세 다이어그램



책임 요약

영역	책임
User/Auth	OAuth 사용자 정보, JWT 발급, 온보딩, 프로필
Refrigerator	재고 수기 등록, OCR/바코드 분석, 목록/상세/수정/삭제
Meal	음식 검색, 식단 저장, 일일 요약, 추천 이력 저장
Recommendation Provider	사용자 냉장고 재료 기반 레시피 추천, mock/live/fallback 위임
Image Analysis Provider	OCR/바코드 입력을 표준 재고 후보 응답으로 변환
Notification	유통기한/공지 알림 생성, 조회, 읽음 처리
Admin	운영 지표, API 사용량, 배치 이력, 공지, 감사 로그

보강 기준

보강 항목	반영 내용
실제 구현 관계 정정	<code>MockRecipeRecommendationProvider</code> , <code>GmsRecipeRecommendationProvider</code> 는 인터페이스 직접 구현체가 아니라 <code>ConfigurableRecipeRecommendationProvider</code> 가 위임 호출하는 provider로 표시
외부 연동 경계 명확화	<code>InventoryImageAnalysisProvider</code> , <code>BarcodeDecoder</code> , <code>FoodSafetyProductClient</code> , OCR client를 분리해 OCR/바코드 연동 경계를 표시
사용량 로깅 반영	OCR/바코드/레시피 추천이 <code>ApiUsageService</code> 를 통해 관리자 사용량 지표로 이어지는 관계 표시
도메인 소유 관계 표현	<code>User -> RefrigeratorItem</code> , <code>User -> MealLog</code> , <code>User -> Notification</code> , <code>MealLog -> MealLogItem</code> 관계를 핵심 도메인 관계로 표시
발표 가독성 조정	기본 다이어그램은 한 화면에서 읽히도록 유지하고, 복잡한 provider 구조는 보조 다이어그램으로 분리

주요 협력 흐름

흐름	클래스 협력
레시피 추천 생성	<code>MealLogController</code> → <code>MealLogService</code> → <code>RecipeRecommendationProvider</code> → <code>ConfigurableRecipeRecommendationProvider</code> → <code>MockRecipeRecommendationProvider</code> 또는 <code>GmsRecipeRecommendationProvider</code> → <code>MealLogDao</code> → <code>ApiUsageService</code>
OCR 분석	<code>RefrigeratorItemController</code> → <code>InventoryImageAnalysisService</code> → <code>InventoryImageAnalysisProvider</code> → <code>FoodSafetyInventoryImageAnalysisProvider</code> → <code>PororoOcrClient</code> → <code>GmsOcrBatchTextClient</code>
바코드 분석	<code>RefrigeratorItemController</code> → <code>InventoryImageAnalysisService</code> → <code>InventoryImageAnalysisProvider</code> → <code>FoodSafetyInventoryImageAnalysisProvider</code> → <code>BarcodeDecoder</code> → <code>FoodSafetyProductClient</code>
유통기한 알림	<code>ExpirationNotificationScheduler</code> → <code>NotificationService</code> → <code>NotificationDao</code>
관리자 대시보드	<code>AdminController</code> → <code>AdminService</code> → <code>AdminDao</code>

품질 보강 포인트

- 추천 생성은 프론트가 외부 AI를 직접 호출하지 않고 백엔드에서 처리하므로 API key 노출 위험을 줄인다.
- `ConfigurableRecipeRecommendationProvider`가 integration mode를 결정하고 mock/live provider를 위임 호출해 데모 환경과 실제 API 환경을 같은 인터페이스로 유지한다.
- OCR/바코드 분석은 `InventoryImageAnalysisProvider` 인터페이스 뒤에 감춰져 있어 provider 교체와 fallback 처리가 쉽다.
- `ApiUsageService`를 공통 로깅 경계로 두어 사용자 기능과 관리자 지표가 느슨하게 연결된다.
- 관리자 기능은 사용자 기능과 분리된 인증/권한 흐름을 가진다.

ER 다이어그램

사용자, 냉장고, 식단, 알림, 관리자 테이블 관계와 데이터 흐름

프로젝트
YumYum

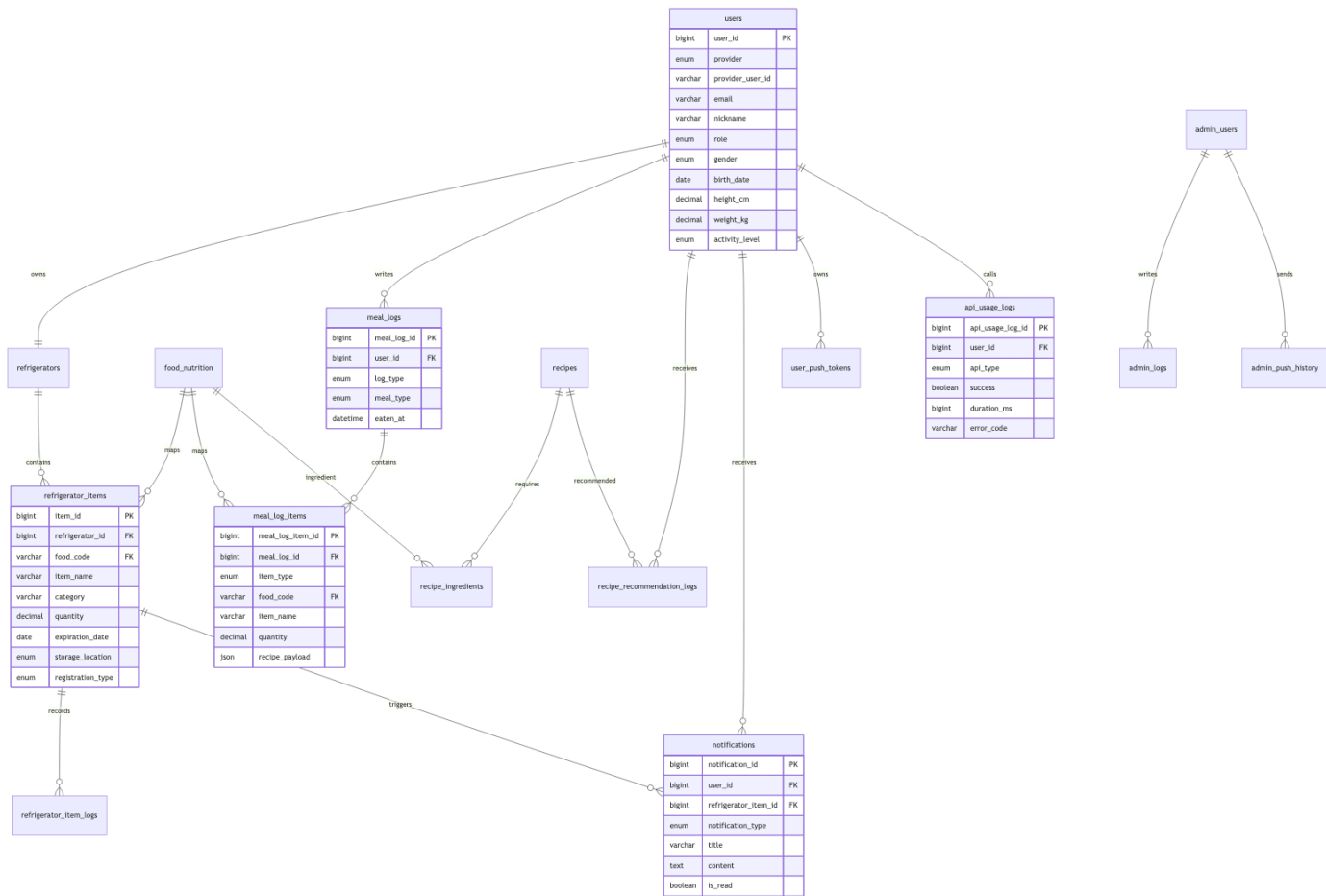
기준 커밋
a963b9d

생성일
2026. 06. 25.

주요 테이블

영역	테이블
사용자	users , admin_users
냉장고	refrigerators , refrigerator_items , refrigerator_item_logs
음식/식단	food_nutrition , meal_logs , meal_log_items
레시피	recipes , recipe_ingredients , recipe_recommendation_logs
알림	notifications , user_push_tokens
운영	api_usage_logs , batch_job_history , admin_logs , admin_push_history

ERD Mermaid 원본



ER 다이어그램 다이어그램 1

설계 포인트

- 식단 기록과 레시피 추천 이력을 `meal_logs.log_type` 과 `meal_log_items.item_type` 으로 구분해 같은 기록 속에서 관리한다.

- 수기 등록 식재료는 `food_code` 가 없을 수 있으므로 `refrigerator_items.food_code` 는 nullable이다.
- 유통기한 알림 중복 생성을 막기 위해 `notifications` 에 사용자, 재고, 상태, 기준일 조합의 unique key를 둔다.
- 외부 API 호출량은 `api_usage_logs` 에 저장해 관리자 화면에서 OCR/바코드/레시피 추천 사용량을 확인한다.

테이블별 역할

테이블	역할	발표 강조점
<code>users</code>	OAuth 사용자와 온보딩 정보를 저장	추천/식단 개인화의 기준 데이터
<code>refrigerator_items</code>	사용자의 실제 재고 상태 저장	유통기한, 보관 위치, 등록 방식 관리
<code>food_nutrition</code>	음식 검색과 영양 계산의 기준	식단 요약 계산의 근거 데이터
<code>meal_logs</code>	식단 기록과 추천 이력의 상위 기록	소비 기록과 추천 결과를 같은 타임라인으로 관리
<code>meal_log_items</code>	식단 음식 또는 추천 recipe payload 저장	식단 상세와 추천 결과 저장을 유연하게 처리
<code>notifications</code>	유통기한/공지 알림 저장	사용자가 놓치기 쉬운 재료 상태를 알림으로 연결
<code>api_usage_logs</code>	OCR/바코드/추천 호출 이력 저장	외부 API 비용과 장애 상황을 운영자가 확인
<code>batch_job_history</code>	유통기한 배치 실행 이력 저장	수동/스케줄 배치 결과를 관리자 화면에 표시

무결성과 인덱스 전략

- 사용자 탈퇴 또는 재고 삭제 이후에도 운영 로그를 보존해야 하는 영역은 `ON DELETE SET NULL` 또는 별도 로그 테이블로 분리한다.
- 음식명 검색과 레시피명 검색은 fulltext index를 사용해 발표 시 “검색 기반 입력 편의성”의 근거로 설명한다.
- 유통기한 알림은 같은 재고에 동일 기준일 알림이 중복 생성되지 않도록 unique key로 방어한다.
- API 사용량 집계는 `created_at` , `api_type` 기준 조회가 잦으므로 복합 index를 둔다.

데이터 흐름 예시

1. 사용자가 냉장고에 `우유` 를 등록하면 `refrigerator_items` 에 저장된다.
2. 배치가 유통기한을 확인해 `notifications` 에 임박 알림을 생성한다.
3. 사용자가 식단 기록에서 `김밥` 을 저장하면 `meal_logs` , `meal_log_items` 에 기록된다.
4. 추천 생성 시 서버는 `refrigerator_items` 를 읽고 추천 결과를 `meal_logs` / `meal_log_items` 에 `RECOMMENDATION` 타입으로 저장한다.
5. OCR/바코드/추천 호출은 `api_usage_logs` 에 기록되어 관리자 대시보드에서 확인된다.

YUMMYUM 설계 문서

WBS & 간트 차트

기획, 구현, 안정화, 문서화 단계별 일정과 리스크 대응

프로젝트

YumYum

기준 커밋

a963b9d

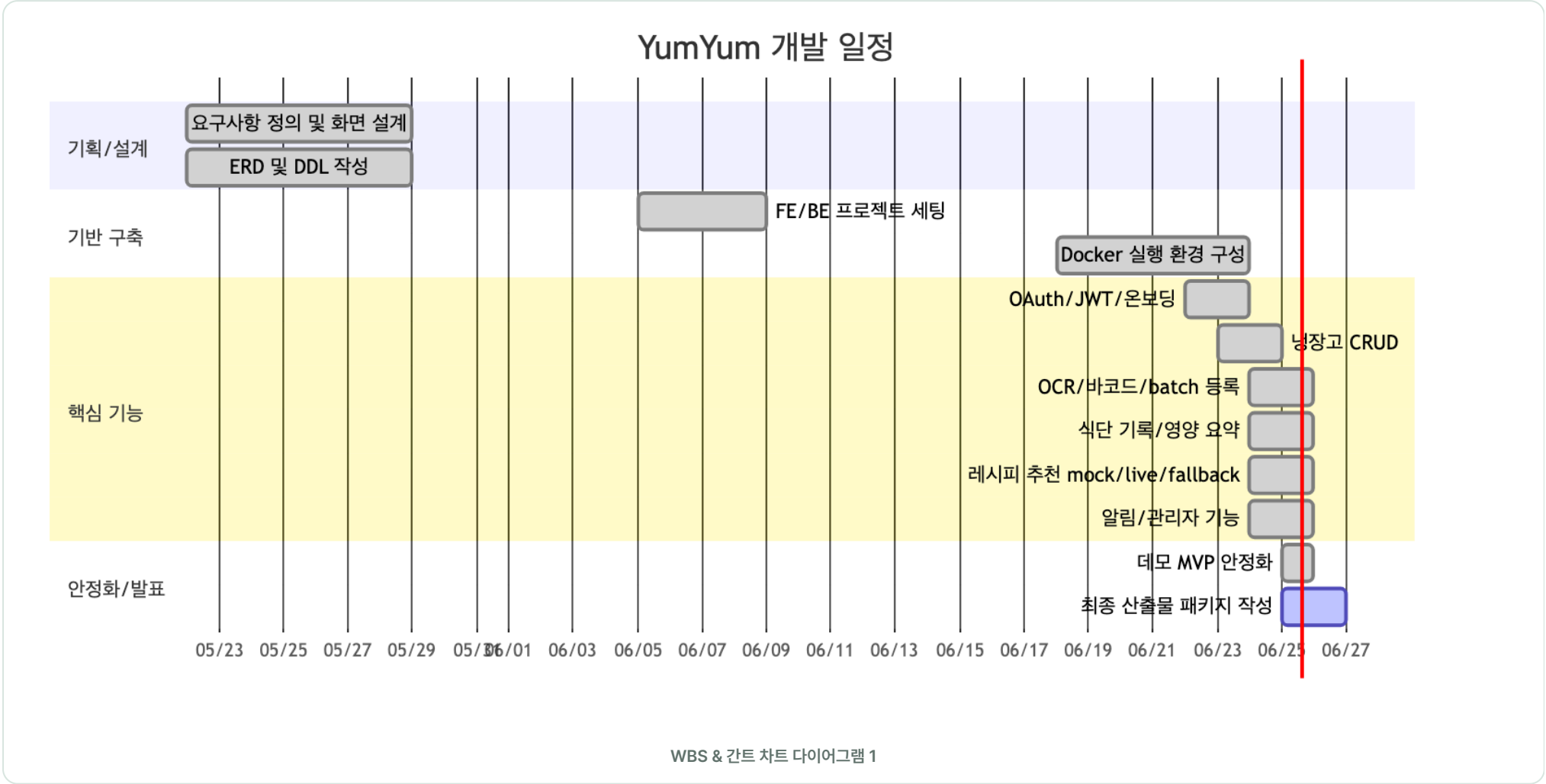
생성일

2026. 06. 25.

WBS

단계	작업	산출물
1. 기획	문제 정의, 기능 범위 선정, 화면 흐름 설계	요구사항 정의서, 화면 설계
2. 데이터 설계	ERD 작성, DDL 작성, 초기 데이터 준비	DB Schema.sql, 초기 데이터
3. 프로젝트 세팅	Vue/Vite FE, Spring Boot BE, Docker Compose 구성	통합 실행 환경
4. 인증/사용자	OAuth2, JWT, 온보딩, 프로필	로그인/온보딩 흐름
5. 냉장고	수기 등록, OCR/바코드 분석, CRUD	재고 관리 화면/API
6. 식단/추천	음식 검색, 식단 저장, 레시피 추천 provider	식단 기록/추천 화면
7. 알림/관리자	유통기한 배치, 알림, 관리자 콘솔	운영 화면
8. 안정화	mock/fallback 모드, 설정 정리, 빌드 검증	데모 MVP
9. 최종 산출물	설계문서, 발표자료 프롬프트, AI 사용 보고서	제출 패키지

Mermaid Gantt 원본



역할별 일정 플레이스홀더

이름	역할	주요 담당	개인 회고 입력 위치
[팀원 A]	PM/QA	요구사항, 시연 흐름, 최종 산출물	[개인 회고]
[팀원 B]	Frontend	Vue 화면, 상태관리, UX	[개인 회고]
[팀원 C]	Backend	Spring Boot, DB, 인증, API	[개인 회고]
[팀원 D]	AI/OCR/Infra	OCR, 외부 API, Docker	[개인 회고]

마일스톤 기준

마일스톤	완료 기준	근거 자료
M1. 기획/설계 확정	요구사항, ERD, 화면 설계 초안 완료	개발 일지 2026-05-22, 2026-05-29
M2. 프로젝트 기반 구축	FE/BE 구조와 통합 repo 구성	개발 일지 2026-06-05
M3. 인증과 사용자 흐름	OAuth, JWT, 온보딩, 프로필 구현	개발 일지 2026-06-22
M4. 핵심 MVP 연결	냉장고, 알림, 추천, FE 연동	개발 일지 2026-06-23
M5. 입력 자동화/인프라	OCR, 바코드, Docker 통합	개발 일지 2026-06-24
M6. 데모 안정화	mock/fallback, 빌드 검증, 화면 캡처	커밋 a963b9d, docs/final-report

리스크와 대응

리스크	영향	대응
외부 API 키 누락	추천/OCR 시연 중단	local/Docker 기본값을 mock으로 설정
OCR 모델 cold start	첫 요청 지연	OCR 서비스 분리와 캐시 볼륨 사용
OAuth redirect 설정 오류	로그인 시연 실패	발표 전 provider 1개와 테스트 계정으로 리허설
팀 정보/사진 누락	최종 PPT 완성도 하락	<code>photos/README.md</code> 플레이스홀더 기준으로 제출 전 교체
발표자료 AI 환각	실제 구현과 다른 슬라이드 생성	<code>slide-outline.md</code> 와 스크린샷 파일명을 함께 제공하고 검수

YUMMYUM 설계 문서

화면 설계서

사용자/관리자 화면 흐름, 캡처 기준, 주요 화면별 설계 의도

프로젝트

YumYum

기준 커밋

a963b9d

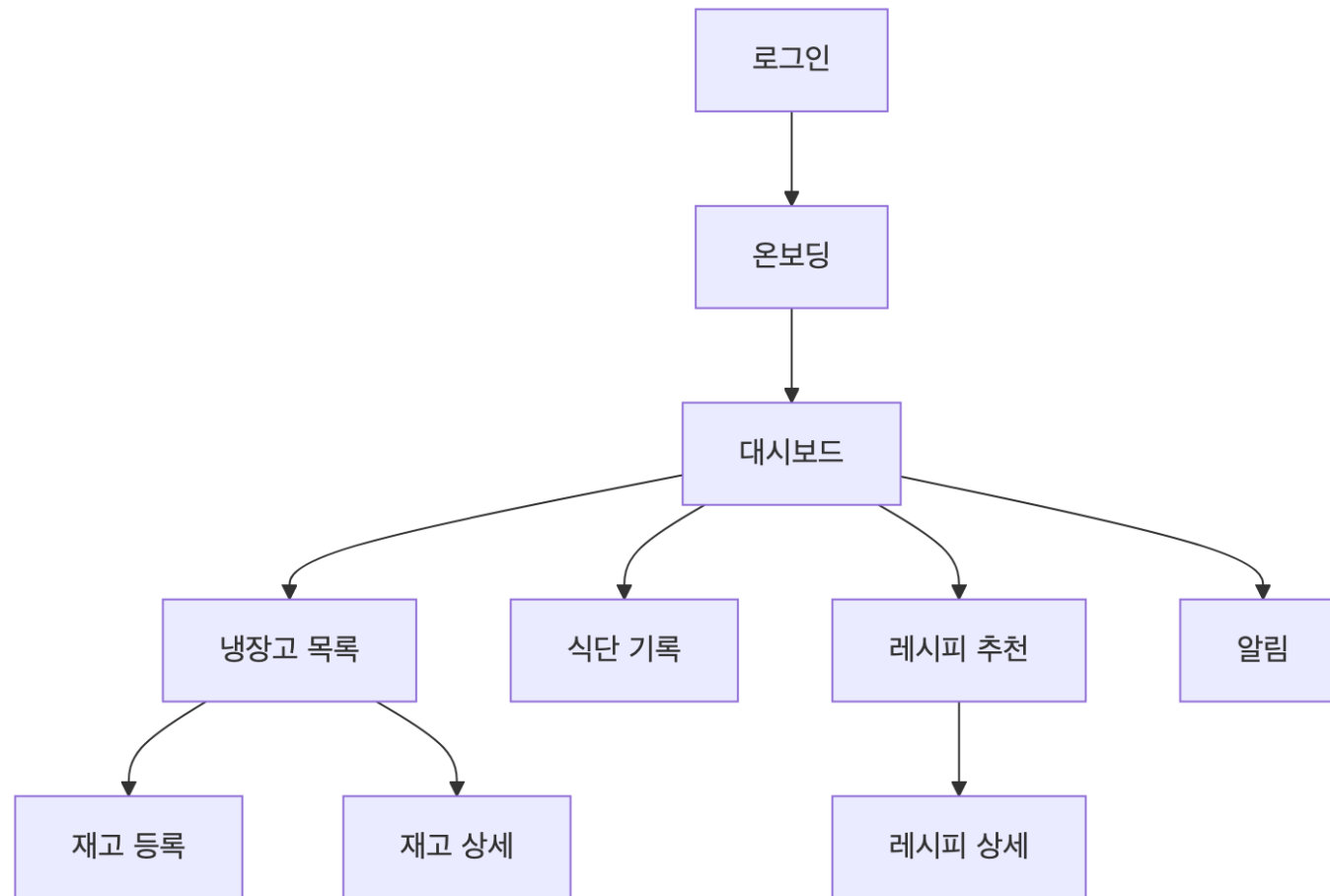
생성일

2026. 06. 25.

화면 목록

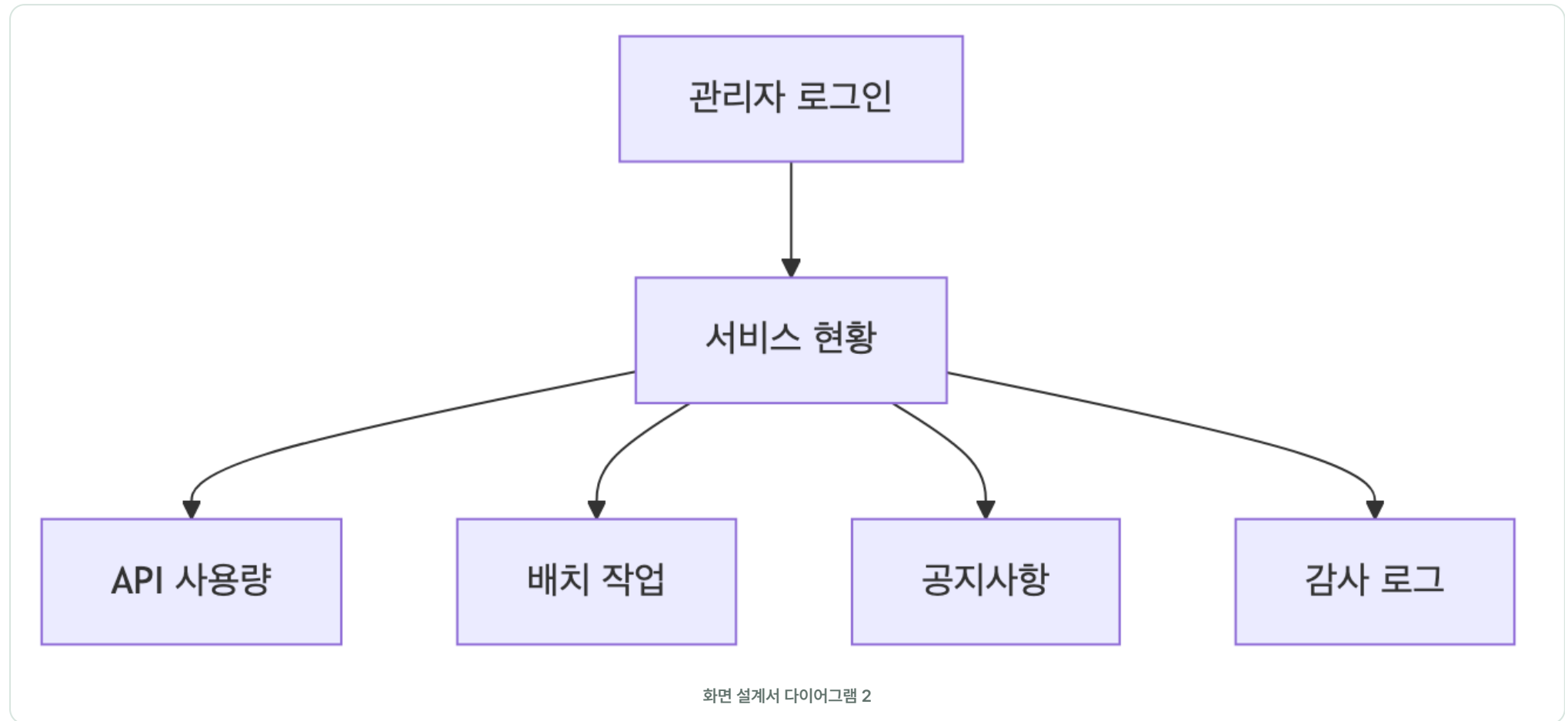
화면	목적	주요 요소	캡처 파일명
로그인	OAuth 로그인 진입	서비스 소개, Google/Naver/Kakao 버튼	01-login.png
온보딩	신규 사용자 정보 입력	닉네임, 성별, 생년월일, 키/몸무게, 활동량	02-onboarding.png
대시보드	서비스 현황 요약	재고 통계, 최근 재고, 알림, 추천 레시피	03-dashboard.png
냉장고 목록	재고 조회/필터	카테고리 필터, 유통기한 상태, 항목 카드	04-inventory-list.png
재고 등록	수기/OCR/바코드 등록	등록 방식, 다중 입력 폼, 분석 결과	05-inventory-add.png
재고 상세	개별 재고 확인/수정	수량, 위치, 유통기한, 수정/삭제	06-inventory-detail.png
식단 기록	음식 검색/저장	날짜, 끼니, 음식 검색, 영양 요약	07-meal-log.png
레시피 추천	재고 기반 추천	추천 카드, 매칭률, 부족 재료, 조리 단계	08-recipe-recommendation.png
알림	알림 조회/처리	유통기한/공지 알림, 읽음 처리	09-notifications.png
관리자 로그인	운영자 인증	아이디/비밀번호 로그인	10-admin-login.png
관리자 대시보드	운영 지표 확인	사용자 수, API 호출, 최근 배치	11-admin-dashboard.png
관리자 API/배치	운영 기능	API 사용량 차트, 배치 실행, 공지 발송	12-admin-operations.png

사용자 화면 흐름



화면 설계서 다이어그램 1

관리자 화면 흐름



화면 설계 원칙

- 발표자료에서는 기능 설명보다 사용자가 실제로 수행하는 흐름을 먼저 보여준다.
- 재고 등록은 **수기 입력**, **OCR**, **바코드** 3가지 진입을 한 화면에서 비교 가능하게 보여준다.
- 식단 기록은 **음식 검색 -> 항목 추가 -> 저장 -> 일일 요약 갱신**의 결과를 한 장에 담는다.

- 레시피 추천은 외부 API 모드보다 사용자가 얻는 결과, 즉 매칭률/활용 재료/부족 재료 중심으로 설명한다.
- 관리자 화면은 데모 안정성, API 비용/장애 관리, 운영 가능성을 뒷받침하는 근거로 사용한다.

화면별 발표 메시지

화면	발표 메시지
로그인	“회원가입 폼 없이 OAuth로 빠르게 시작합니다.”
온보딩	“추천과 영양 요약에 필요한 최소 개인 정보를 입력합니다.”
대시보드	“재고 상태, 알림, 추천 결과를 한 화면에서 확인합니다.”
냉장고 목록	“재고가 유통기한 상태와 함께 정리되어 낭비를 줄입니다.”
재고 등록	“OCR/바코드/수기 입력을 하나의 등록 경험으로 묶었습니다.”
식단 기록	“먹은 음식과 영양 합계를 한 화면에서 관리합니다.”
레시피 추천	“냉장고에 있는 재료를 바로 조리 아이디어로 연결합니다.”
알림	“임박/만료/공지 알림으로 사용자의 다음 행동을 유도합니다.”
관리자	“PM/운영자가 API 사용량과 배치 상태를 확인할 수 있습니다.”

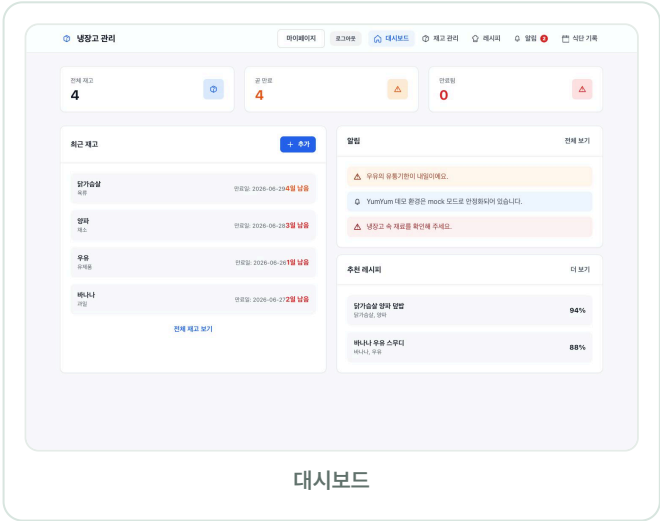
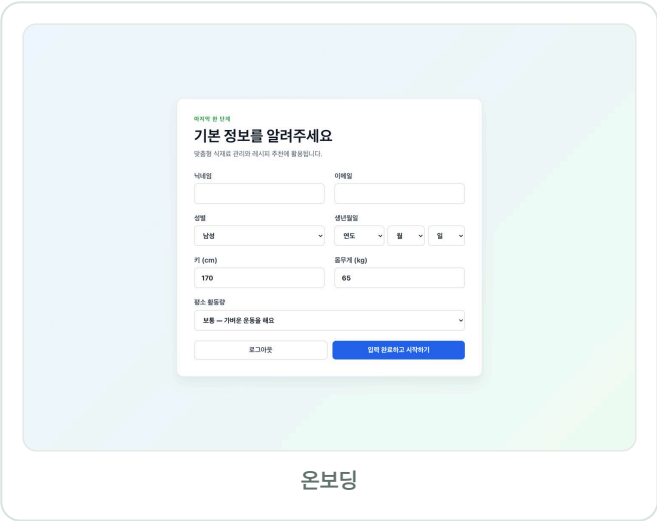
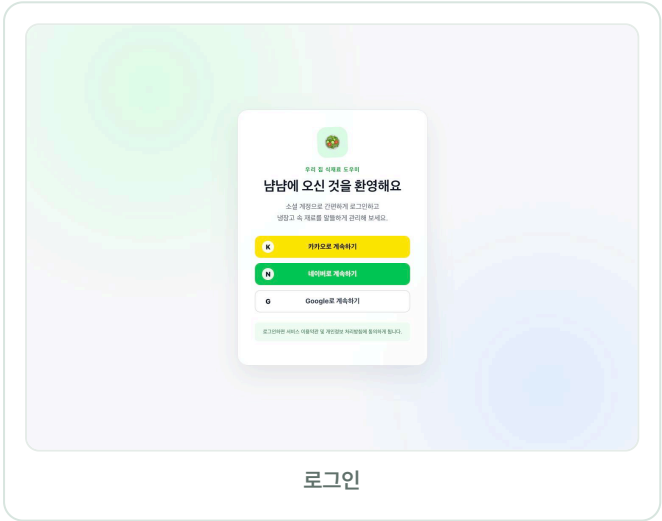
화면 상태 기준

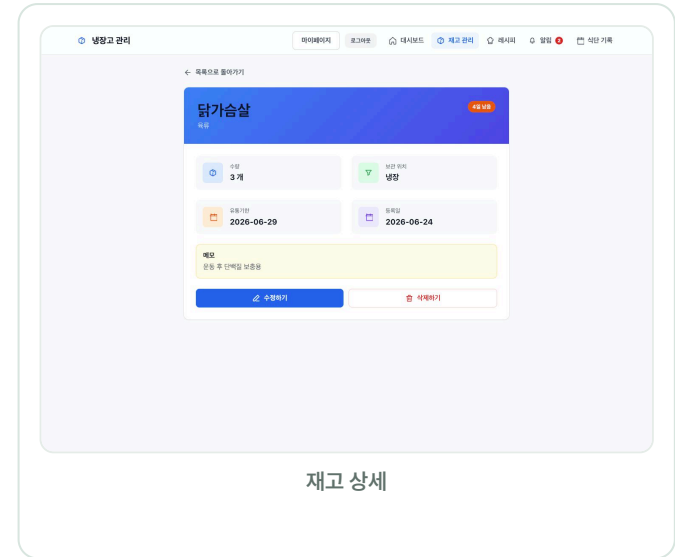
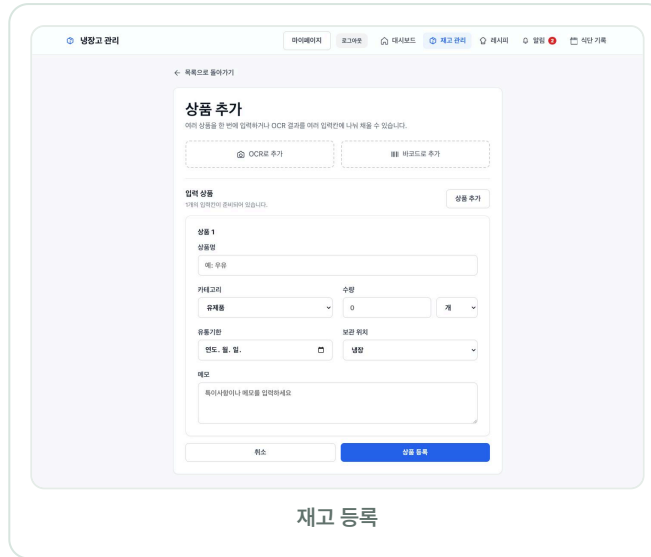
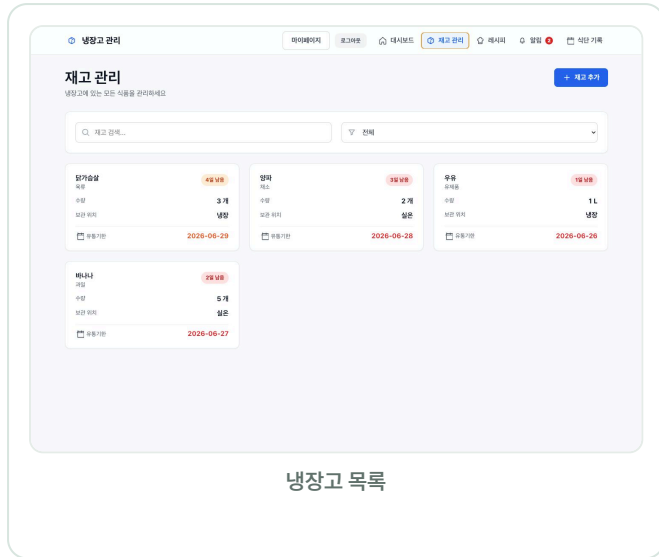
- 정상 상태: 데이터가 있는 대표 캡처를 발표자료 기본 이미지로 사용한다.
- 빈 상태: 기능 설명보다 “처음 시작하는 사용자도 이해 가능”한 onboarding/empty UX 보조 자료로만 사용한다.
- 오류 상태: 발표 본문에는 넣지 않고, Q&A나 기술 설명에서 fallback 전략을 설명할 때만 사용한다.
- 관리자 화면: 실제 계정 정보가 아닌 더미 사용자와 더미 관리자 계정 캡처만 사용한다.

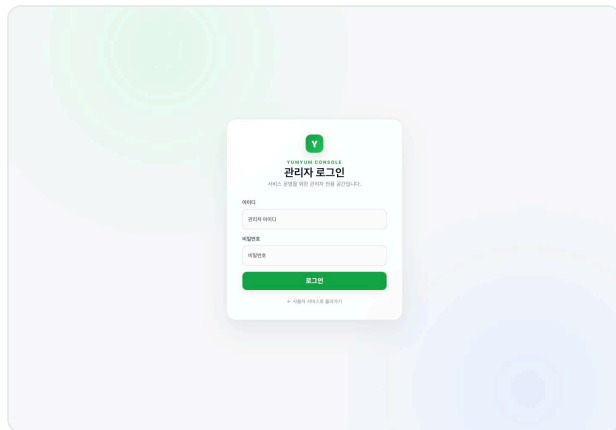
캡처 품질 기준

- 화면 내 텍스트가 잘리지 않아야 한다.
- 브라우저 주소창과 개발자 도구는 포함하지 않는다.
- 발표 슬라이드에서 16:9 크롭이 가능하도록 핵심 정보는 화면 중앙 또는 상단에 배치된 캡처를 사용한다.
- 개인정보처럼 보이는 이메일, 토큰, client secret, API key는 캡처에 포함하지 않는다.

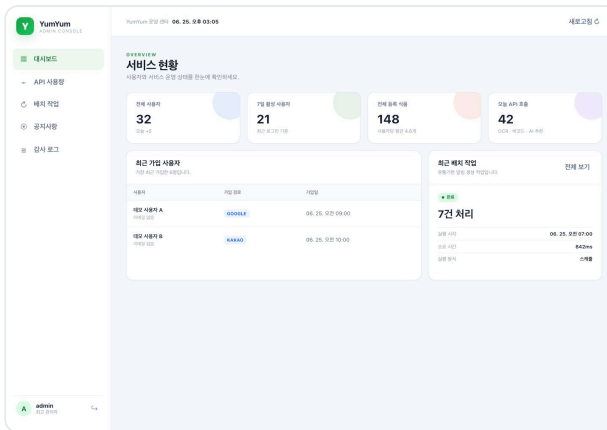
화면 캡처 자료



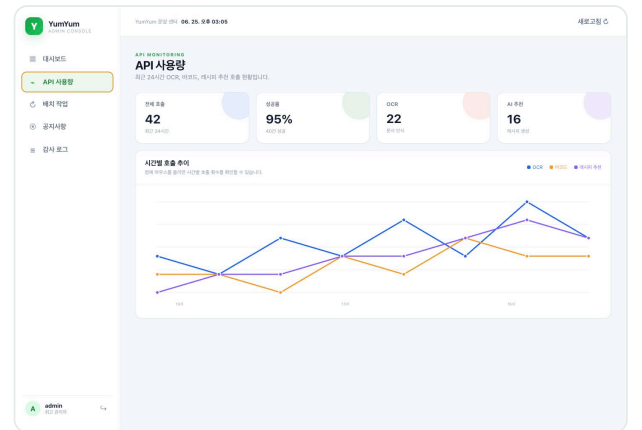




관리자 로그인



관리자 대시보드



관리자 운영 화면

기타 참고 문서

실행 환경, 주요 API, 검증 명령, 제출 전 체크리스트

프로젝트

YumYum

기준 커밋

a963b9d

생성일

2026. 06. 25.

개발 일지 기반 프로젝트 흐름

날짜	핵심 내용
2026-05-22	ERD와 DDL 초안 작성. 냉장고, 식단, 추천, 알림, 관리자 도메인 정의
2026-05-29	기능/화면 기획. OCR/바코드/수기 등록, 추천 기준, 유통기한 배치 범위 결정
2026-06-05	Vue/Vite FrontEnd와 Spring Boot BackEnd 프로젝트 구조 착수
2026-06-22	Spring Security, OAuth2, JWT, 공통 응답/예외, 사용자 도메인 구현
2026-06-23	OAuth/온보딩, 냉장고 CRUD, 알림, 레시피 추천 이력, FE 연동
2026-06-24	OCR/바코드/GPT 구조화, FastAPI OCR 서비스, batch 등록, Docker 환경 구성
2026-06-25	데모 MVP 안정화. mock/live/fallback 통합 모드, 서버 추천 생성 API, 설정 정리

실행 환경

- FrontEnd: Vue 3, Vite, Pinia
- BackEnd: Spring Boot, MyBatis, MySQL
- OCR Service: FastAPI, Pororo OCR
- Infra: Docker Compose
- 외부 연동: OAuth Provider, GMS/GPT, 식품안전나라 API

주요 API

영역	API
사용자	<code>GET/PATCH /api/users/me</code> , <code>PATCH /api/users/me/onboarding</code> , <code>PATCH /api/users/token/refresh</code>
냉장고	<code>GET /api/refrigerator/items</code> , <code>POST /api/refrigerator/items/manual</code> , <code>POST /api/refrigerator/items/analyze/{analysisType}</code>
식단	<code>GET /api/meal-logs/foods/search</code> , <code>POST /api/meal-logs</code> , <code>GET /api/meal-logs/summary</code> , <code>DELETE /api/meal-logs/{id}</code>
추천	<code>POST /api/meal-logs/recommendations/generate</code> , <code>GET /api/meal-logs/recommendations/latest</code>
알림	<code>GET /api/notifications</code> , <code>PUT /api/notifications/{id}/read</code> , <code>PUT /api/notifications/read-all</code>
관리자	<code>POST /api/admin/auth/login</code> , <code>GET /api/admin/dashboard</code> , <code>GET /api/admin/api-usage</code> , <code>POST /api/admin/batches/expiration</code> , <code>POST /api/admin/notices</code>

제출 파일 구성 제안

- 설계 문서: `01-design-doc/` 원본을 Word 또는 PDF로 변환
- 소스 코드: `BackEnd`, `FrontEnd`, `OcrService`, `docker-compose.yml`, SQL 파일을 zip으로 제출
- 발표자료: Gamma에서 생성한 PPT/PDF
- AI 사용 보고서: `03-ai-report/` 원본을 별도 PDF 또는 발표 부록으로 제출

검증 명령 기록

검증	명령	상태
FrontEnd build	<code>npm run build</code>	통과
BackEnd compile	<code>mvn -q -DskipTests compile</code>	통과
Compose config	<code>docker compose config --quiet</code>	통과
문서 whitespace	<code>git diff --check</code>	통과
비밀값 패턴 검색	<code>rg</code> 기반 token/secret/API key 검색	매칭 없음

외부 AI에 전달할 때 주의할 점

- `gamma-prompt.md` 만 단독으로 넣기보다 `slide-outline.md`, `speaker-notes.md`, `screenshots/README.md` 를 함께 제공한다.
- Mermaid 원본은 Gamma가 직접 렌더링하지 못할 수 있으므로, 필요하면 별도 Mermaid renderer에서 PNG로 변환한다.
- 팀명, 팀원, 팀 사진, 개인 회고는 실제 정보로 교체하기 전까지 플레이스홀더로 유지한다.
- 외부 AI가 실제 구현과 다른 기능을 추가하면 삭제한다. 특히 "모바일 앱", "실시간 푸시 운영", "정식 배포 URL"은 이번 제출 범위 밖이다.

제출 전 마지막 체크리스트

- ☐ 설계문서 6개 항목이 모두 Word/PDF에 포함됨
- ☐ PPT가 최소 10장 이상이며 20분 발표 흐름을 가짐
- ☐ AI 사용 보고서가 별도 파일 또는 PPT 부록에 포함됨
- ☐ 화면캡처 12장이 발표자료에 연결됨
- ☐ 팀 정보와 사진 플레이스홀더가 실제 정보로 교체됨
- ☐ 문서와 이미지에 secret/token/API key가 없음